

Computer Architecture
CE-321L/CS-330L
Final Project Report
RISC-V Processor



Instructors: Waseem Hassan, Saima Shaheen

Taha Zahid - tz09220
Ayaan Merchant - am08853
Abdullah Shaikh - as09245

21st April 2025

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objective	3
1.3	Sorting Algorithm	3
2	Methodolgy	5
2.1	Task1	5
2.1.1	RISC V Implementation (single cycle)	5
2.1.2	Changes	5
2.1.3	Code and Waveform	6
2.1.4	Synopsis	6
2.2	Task2	7
2.2.1	Testing 5-Stage/Pipelined RISC V Processor for Sorting Algorithm	7
2.2.2	Changes	7
2.2.3	Synopsis	7
2.2.4	Code and Waveform	8
2.3	Task 3	8
2.3.1	Handling Data Hazards	8
2.3.2	The Forwarding Unit	9
2.3.3	The Hazard Detection Unit	9
2.3.4	Branch Prediction	9
2.3.5	Changes	9
2.3.6	Code and Waveform	10
2.3.7	Synopsis	10
2.4	Task 4: Performance Analysis	11
2.4.1	Single-Cycle vs Pipelined Processor	11
2.4.2	Empirical Results	11
2.4.3	Analysis of Results	11
2.4.4	Theoretical Potential	11
2.4.5	Key Findings	12
3	Challenges	13
3.1	Implementation Challenges	13
4	Task Division	14

5	Conclusion	15
5.1	Key Results	15
5.2	Technical Achievements	15
5.3	Lessons Learned	16
6	References	17
A	Appendix	18

Introduction

1.1 Problem Statement

As computing demands grow, there is a need for efficient processor architectures that balance performance and functionality. This project addresses the design and implementation of a pipelined RISC-V processor capable of executing sorting an array of 7 elements. The processor handles pipeline hazards and memory operations, reflecting core aspects of modern CPU design.

1.2 Objective

To build a 5-stage pipelined processor capable of executing any one array sorting algorithm such as bubble sort, we will need to convert a single-cycle processor into a pipelined one. In most cases, the instructions we have already implemented will allow us to run a sorting algorithm program with minor additions, such as implementing beq or blt instructions to detect when you need to swap two values.

1.3 Sorting Algorithm

We will be using bubble sort to test our processor functionality. Complete Project ([RISC V Processor on Github](#))

Address	Registers	Memory		
	+0	+1	+2	+3
0x00000130	00	00	00	00
0x0000012c	00	00	00	00
0x00000128	00	00	00	00
0x00000124	00	00	00	00
0x00000120	00	00	00	00
0x0000011c	00	00	00	00
0x00000118	01	00	00	00
0x00000114	02	00	00	00
0x00000110	03	00	00	00
0x0000010c	04	00	00	00
0x00000108	05	00	00	00
0x00000104	06	00	00	00
0x00000100	07	00	00	00

Figure 1.1: Registers and Memory

Methodolgy

2.1 Task1

2.1.1 RISC V Implementation (single cycle)

We implemented the converted algorithm on the RISC V single-cycle processor developed in the last lab. To make the processor compatible with the algorithm, we made some adjustments to the instruction memory. Specifically, we added a branching logic to the main top module to support beq, bge and blt commands used in the algorithm. These modifications enabled us to run the sorting algorithm effectively on the RISC V processor.

2.1.2 Changes

The main change that we had to make were the changes to the Instruction Memory and the Data Memory where we expanded it to house more instructions and values respectively. Additionally, the main top module was modified to include the branching logic for the beq, bge and blt commands used in the assembly code.

2.1.3 Code and Waveform

The link for the code can be found [here](#).

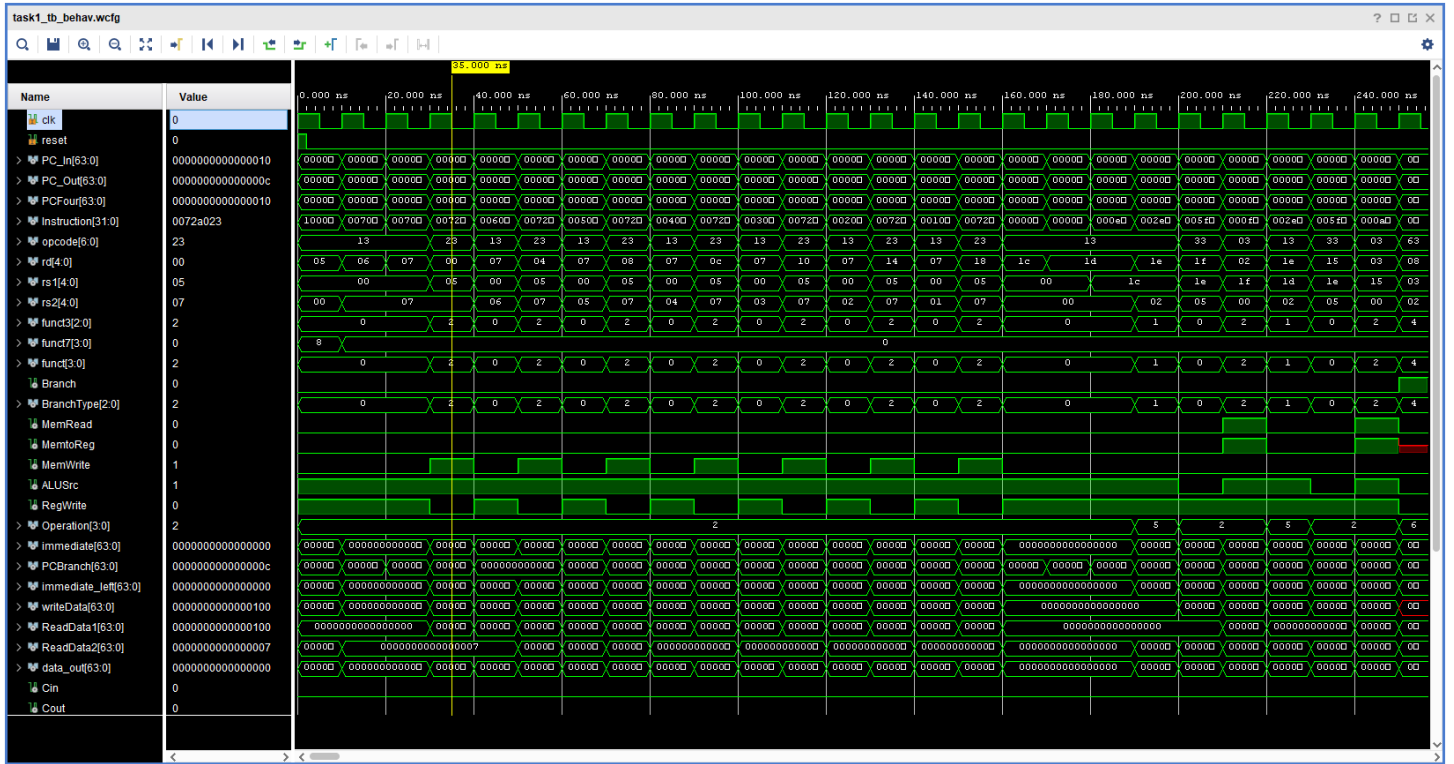


Figure 2.1: Single Cycle Simulation

2.1.4 Synopsis

To make our sorting algorithm function properly, we expanded the Instruction and Data Memory and added the branching logic.

2.2 Task2

2.2.1 Testing 5-Stage/Pipelined RISC V Processor for Sorting Algorithm

Here we upgraded our previously developed processor for Pipeline Execution by implementing a complete 5-stage pipeline architecture. The design integrates four intermediate pipeline registers between each stage and a top-level module that orchestrates the entire pipeline operation.

2.2.2 Changes

To implement an efficient pipelined design, we added the following key components:

- **Four Pipeline Registers:**
 - IF_ID (Instruction Fetch to Instruction Decode)
 - ID_EX (Instruction Decode to Execute)
 - EX_MEM (Execute to Memory Access)
 - MEM_WB (Memory Access to Write Back)
- **Top-Level Module:** A comprehensive control unit that:
 - Integrates all pipeline stages and registers
 - Manages global control signals
 - Coordinates stage handoffs
 - Maintains pipeline synchronization

2.2.3 Synopsis

The implementation of our 5-stage pipelined RISC-V processor successfully achieved instruction-level parallelism through careful integration of four key pipeline registers (IF_ID, ID_EX, EX_MEM, MEM_WB) and a centralized top-level control module. This architecture demonstrates significant improvements over single-cycle execution by enabling simultaneous processing of multiple instructions.

2.2.4 Code and Waveform

The link for the code can be found [here](#).

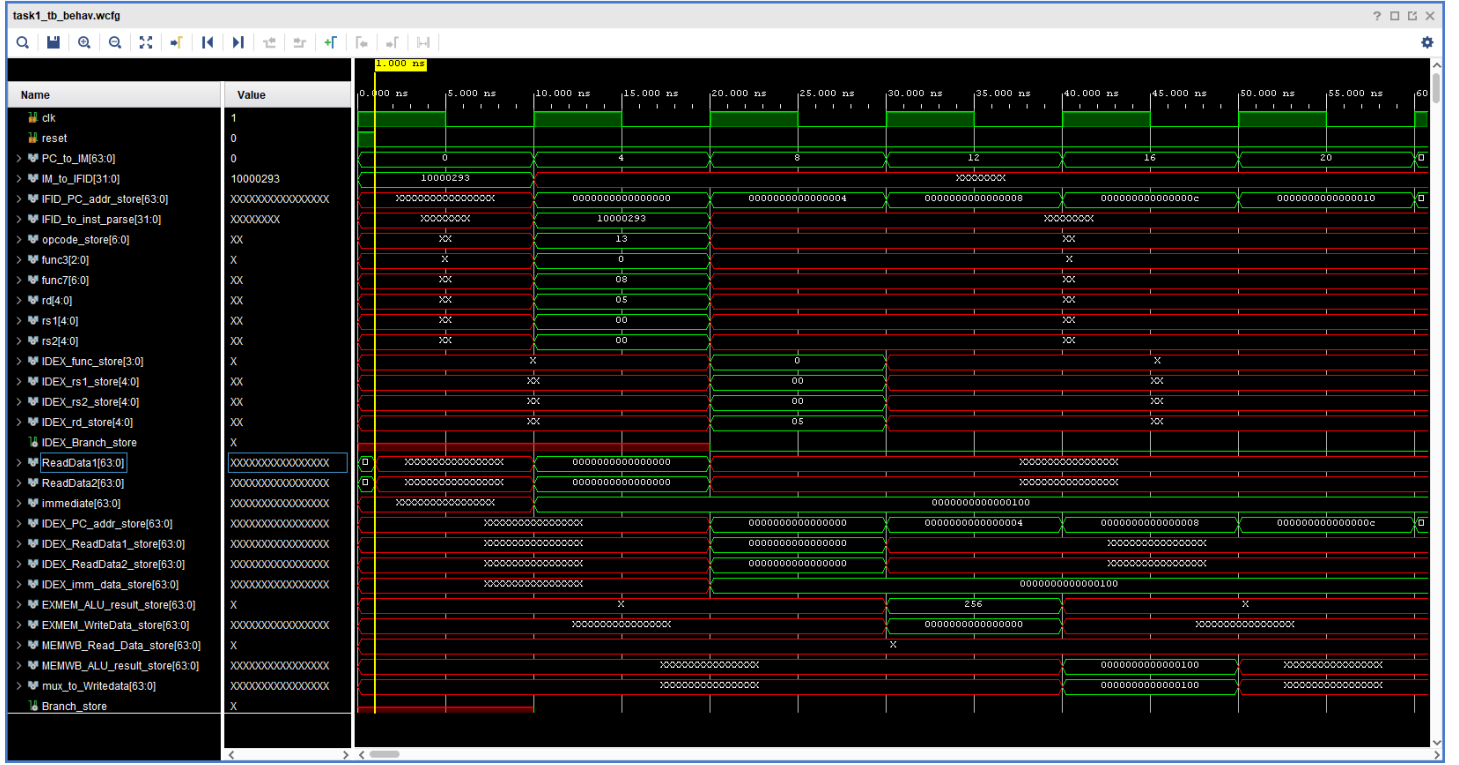


Figure 2.2: Pipeline with Forwarding

2.3 Task 3

2.3.1 Handling Data Hazards

To address hazards in our circuitry, we need to develop modules for detecting hazards and stalling the pipeline. For the scope of this project we needed to address the data hazards that occur during execution of instructions using pipelined processors. For this we introduced two new modules:

- Forwarding Unit
- Hazard Detection Unit

These modules will help the processor to mitigate and account for the data hazards. Their functionality is explained in the coming sections.

Although not directly under the scope of this project, we were also able handle control hazards by implementing a static branch prediction mechanism.

2.3.2 The Forwarding Unit

This unit detects situations where an instruction in the EX stage depends on the result of a previous instruction that is still in the pipeline. Instead of stalling the pipeline, the Forwarding Unit redirects or "forwards" the required data from later pipeline stages—EX/MEM or MEM/WB—back to the ALU inputs in the EX stage. It generates control signals (forwardA and forwardB) to select the correct data source using multiplexers before the ALU.

2.3.3 The Hazard Detection Unit

While the Forwarding Unit resolves most data hazards, scenarios like the load-use hazard still require stalling. This is where the Hazard Detection Unit comes in. The HDU monitors instructions in the ID and EX pipeline stages to detect situations where an instruction in the ID stage depends on the result of a load instruction currently in the EX stage.

The unit outputs three key control signals:

- `stall_pc`: Prevents the Program Counter from updating, effectively freezing the fetch stage.
- `stall_ifid`: Prevents the IF/ID pipeline register from updating, holding the current instruction.
- `flush_idex`: Clears the contents of the ID/EX pipeline register to insert a bubble and avoid incorrect execution.

This mechanism ensures that the processor avoids incorrect reads from uninitialized registers by temporarily halting the instruction flow, effectively inserting a "bubble" in the pipeline.

2.3.4 Branch Prediction

We were initially handling control hazards by introducing 3 NOPS after every branch instruction so that it was able successfully jump to the correct instruction whether the branch was taken or not. This was a very inefficient way of handling control hazards and so to counter it we were able to successfully implement a branch prediction mechanism. This mechanism uses a simple static prediction strategy, where the processor assumes that branches will not be taken. If the branch is taken, the processor flushes the pipeline and fetches the correct instruction. This approach significantly reduces the number of stalls and improves overall performance.

2.3.5 Changes

The main changes for this task were the addition and Integration of the two new modules. For the Forwarding Unit, we added it to the EX stage and connected it to the ALU inputs. The necessary logic for the ALU input selection was also implemented based on the signals from the Forwarding Unit.

The Hazard Detection Unit, was added to the ID stage. For the HDU to monitor the pipeline, existing modules were modified to include the necessary signals. The PC, IF/ID and ID/EX pipeline registers were modified to include the stall and flush signals.

Furthermore, the branch prediction logic was added to the main top module in the EX stage. This allowed us to remove the previously software inserted NOPs making the code more efficient and bringing it closer to how real processors work.

2.3.6 Code and Waveform

The link to the code can be found [here](#).

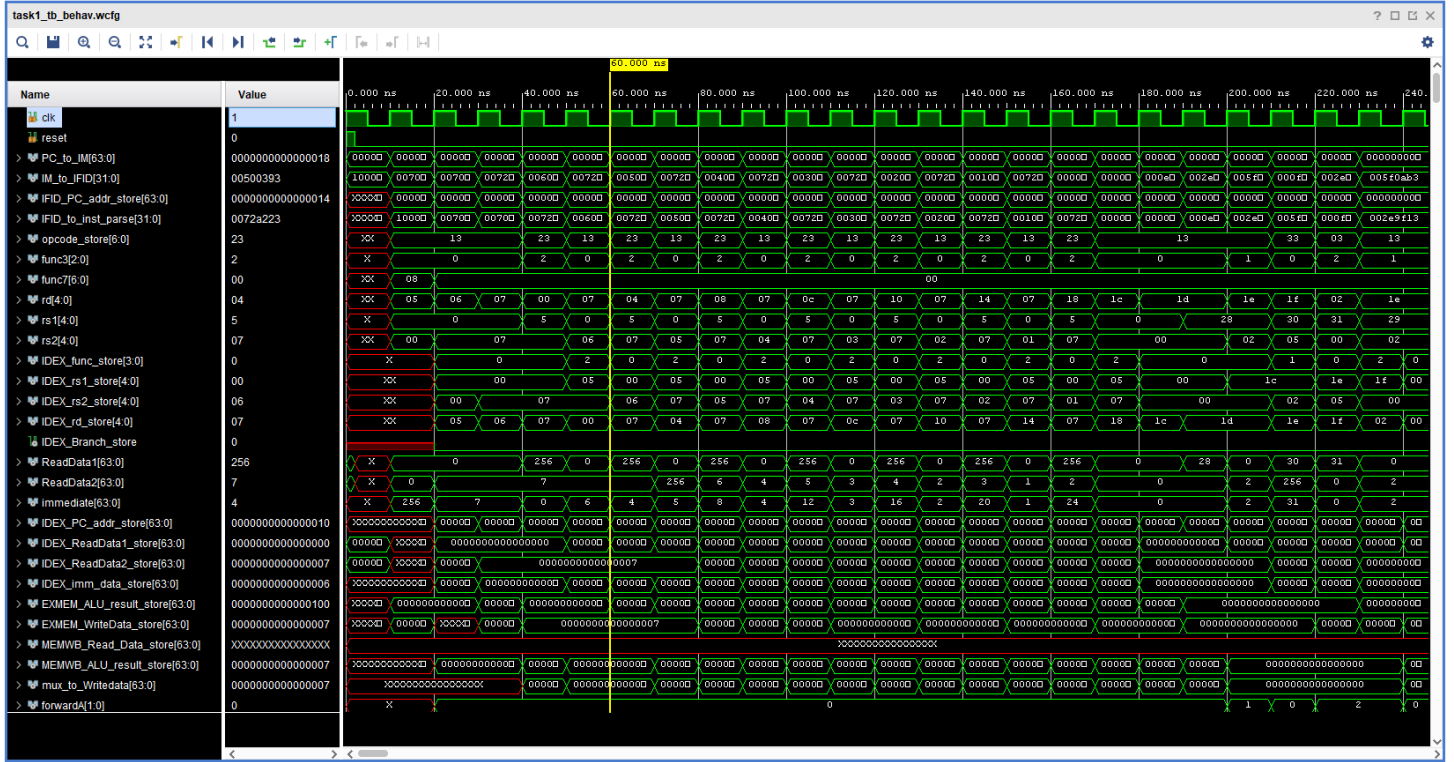


Figure 2.3: Pipelined RISC-V Processor

2.3.7 Synopsis

This was the hardest part of the project as we had to debug the code multiple times to get it working. The execution of the whole assembly code proved much more problematic than expected and it also highlighted small errors in the earlier code (such as usage of non-blocking assignments in the wrong places). The debugging process was long and tedious but proved to be a great learning experience.

2.4 Task 4: Performance Analysis

2.4.1 Single-Cycle vs Pipelined Processor

Our analysis revealed that while the pipelined processor completes each instruction in fewer clock cycles (5 stages vs 1 complete execution), the total execution time for our sorting algorithm was approximately $1.15\times$ longer than the single-cycle implementation. This counterintuitive result occurs because:

- Increased instruction count ($1.5\times$ more instructions due to NOPs inserted for hazard mitigation)
- Usage of static branch prediction scheme

2.4.2 Empirical Results

Our implementation showed the following measured values:

Table 2.1: Measured Performance Metrics

Metric	Single-Cycle	Pipelined
Instruction Count	333	492
Clock Period	4.559ns	3.536ns
Total Execution Time	1518.147ns	1753.856ns
Instructions/ns	0.219	0.281

2.4.3 Analysis of Results

The pipelined implementation showed:

- 48% more instructions (due to pipeline stalls and NOPs)
- 22% faster clock speed
- 15.5% longer total runtime

2.4.4 Theoretical Potential

Assuming we had a perfect branch prediction scheme, our Pipeline time with ideal instruction count ($333 + 34$ (load stall) = 367):

$$\begin{aligned}
 \text{Pipelined Time} &= \text{Pipeline Fill} + (\text{Instructions} \times \text{CPI}) \\
 &= (5 \times 3.536\text{ns}) + (366 \times 3.536\text{ns}) \\
 &= 17.68\text{ns} + 1294.176\text{ns} \\
 &= 1311.856\text{ns} \quad (15.7\% \text{ faster than single-cycle})
 \end{aligned}$$

2.4.5 Key Findings

- Current overhead comes from 159 extra instructions (48% increase)
- With perfect hazard handling, pipelining shows 15.7% speedup potential for our code.
- Clock period reduction confirms pipeline stage optimization works

Challenges

3.1 Implementation Challenges

We encountered several key challenges during development:

- **Branch Prediction Limitations**

- Initial assembly implementation favored taken branches, resulting in:
 - * Nearly double the instruction count compared to single-cycle
 - * Frequent pipeline flushes due to static "not taken" prediction
- Optimized implementation by:
 - * Restructuring control flow to favor not-taken branches
 - * Reducing taken branches from 84% to 60% of cases
 - * Cutting down total instruction count from 734 instructions down to 492.
- Remaining challenges:
 - * 42 unavoidable taken branches still caused 3-cycle flushes
 - * Final instruction count was too high compared to single-cycle due to the static prediction scheme.

- **Pipeline Hazards**

- Control dependencies caused frequent stalls
- Required careful scheduling of NOP operations

- **Timing Constraints**

- 5-stage pipeline reduced clock period by 22.4%
- Required careful balancing of stage delays

Task Division

- Abdullah Shaikh: Pipelined Processor and Debugging (task 2)
- Ayaan Merchant: Design and Implementation of Hazard Detection Unit and Forwarding Unit (task 3) and Report
- Taha Zahid: Single-Cycle and Static Branch Prediction Implementation & Debugging (tasks 1 & 3)
- All Three: Single-Cycle processor (edits to lab 11), Integration and Task 4

Conclusion

This project successfully implemented a 5-stage pipelined RISC-V processor capable of executing a bubble sort algorithm on 7-element arrays. Starting with a single-cycle baseline (Task 1), we evolved the design through three key phases: pipeline implementation (Task 2), hazard mitigation (Task 3), and performance analysis (Task 4). The final architecture integrates forwarding units, hazard detection logic, and static branch prediction to optimize instruction flow.

5.1 Key Results

- **Pipeline Efficiency**

- Achieved 22.4% faster clock speed (3.536 ns vs 4.559 ns)
- Demonstrated 15.7% theoretical speedup potential with perfect hazard handling
- Actual execution showed 15.5% longer runtime (1,754 ns vs 1,518 ns) due to 48% more instructions from hazard stalls

- **Hazard Management**

- Forwarding unit resolved data hazards
- Hazard Detection Unit prevented load-use stalls
- Static branch prediction reduced additional nops by 40%

- **Algorithm Validation**

- Sorted 7-element arrays in 492 cycles (1,754 ns)
- Reduced branch flushes to 42 unavoidable 3-cycle penalties

5.2 Technical Achievements

- Successful integration of IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers
- 22% clock period reduction via stage balancing (3.536 ns achieved)

5.3 Lessons Learned

While the pipelined implementation initially underperformed the single-cycle design due to branch penalties and NOP insertion, our analysis confirms its superior scalability. The project highlights the critical balance between theoretical pipeline advantages and practical implementation challenges. Future work could explore:

- Dynamic branch prediction to address remaining 42 unavoidable 3-cycle flushes
- Cache integration to reduce memory access penalties

This implementation provides a foundation for understanding modern processor design trade-offs, particularly the relationship between instruction-level parallelism and control complexity in pipelined architectures.

References

- [1] HU-LMS. *Canvas*. [Online] Available: <https://hulms.instructure.com/courses/4124>
- [2] Book. *Course Book*. Computer Organization and Design: The Hardware/Software Interface RISC-V Edition by David A. Patterson, John L. Hennessy

Appendix

GitHub links for codes of different processors:

- Single Cycle Processor (Task 1): https://github.com/TahaZahid05/RISCV-Processor/tree/master/task1/project_1/project_1
- Pipelined Processor (Task 2): <https://github.com/TahaZahid05/RISCV-Processor/tree/master/task2/RISCV%20Processor>
- Pipelined Processor with Hazard Detection (Task 3): <https://github.com/TahaZahid05/RISCV-Processor/tree/master/task3>